UNIBALSAS · CENTRO UNIVERSITÁRIO

Workshop · Desenvolvimento de APIs REST e Integração de Sistemas com Laravel

SEGUNDO ENCONTRO

Integração com Pagamentos e Webhooks

```
POST /v3/payments
"status": "PENDING"
POST /api/webhooks/asaas
"event": "PAYMENT_RECEIVED"
```

a API de inscrições passa a conversar com uma plataforma de pagamentos real: gera cobranças no Asaas e recebe, por webhook, o aviso de que o pagamento foi feito.

Integrando a API a um serviço externo, em ambiente de testes

WORKSHOP
APIs REST e Integração com Laravel

PROFESSOR
Alex Rogaleski Marques

CONTATO
alex.rogaleski@prof.unibalsas.edu.br

Tutorial prático e material de projeção em sala · Workshop em dois encontros

1

ABERTURA

Objetivos, retomada e roteiro

No encontro anterior, construímos uma API REST de inscrições: eventos, inscrições, validação, regras de negócio, login e regras de acesso por dono. Ficou uma pendência de propósito: a inscrição em evento pago nasce **pendente**, e a API não tem como cobrar nem como saber que alguém pagou. Hoje resolvemos isso integrando a API ao **Asaas**, uma plataforma brasileira de pagamentos, no seu ambiente de testes. Vamos ver as duas direções de uma integração: a nossa API **chamando** um serviço externo e o serviço externo **chamando** a nossa API.

ANTES DE COMEÇAR

- O projeto do encontro anterior deve estar funcionando, com o login da Etapa 7 e as regras de dono da Etapa 8: `php artisan serve` rodando, login feito no Postman (*Auth — login*) e `GET {{base_url}}/eventos` respondendo `200`, com o campo `user_id` em cada evento. Se der `401`, faça o login de novo. Os exercícios não são pré-requisito.
- Tenha em mãos a **chave de API do sandbox** (`$aact_hmlg_…`) e o acesso a `https://sandbox.asaas.com/login/auth`, conforme o material prévio.
- Como antes, as respostas mostradas são reais. Os identificadores do Asaas (`cus_…`, `pay_…`, `evt_…`) e as datas da sua execução serão outros.

Objetivos de aprendizagem

Ao final deste encontro, espera-se que o participante seja capaz de:

- Ler a documentação de uma API de terceiros e testá-la no Postman antes de programar.
- Consumir uma API externa com o cliente HTTP do Laravel.
- Explicar o que é um webhook e por que ele substitui a consulta repetida.
- Expor a API local à internet com um túnel para testar a integração de ponta a ponta.
- Guardar credenciais com segurança em variáveis de ambiente.
- Tratar falhas do serviço externo sem deixar dados inconsistentes.
- Receber webhooks com autenticação por token e processamento idempotente.
- Testar a integração automaticamente, simulando o serviço externo.

O fluxo que vamos construir

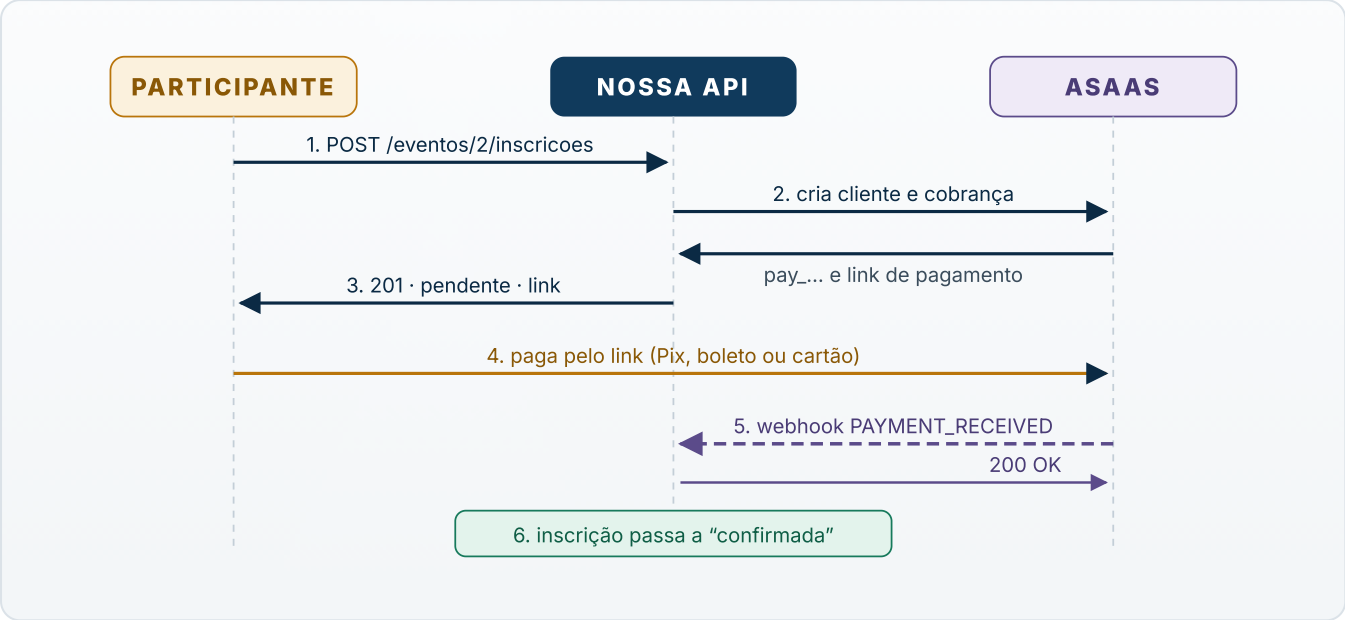


Figura 1.1 — Fluxo completo da inscrição paga. Os passos 2 e 3 são a *integração de saída*; os passos 5 e 6, a *integração de entrada*, por webhook.

Roteiro do encontro

1	**Conhecendo a API do Asaas.** Autenticação, clientes e cobranças, primeiro no Postman.
2	**Credenciais.** A chave no `.env` e a configuração do serviço.
3	**Cliente HTTP.** Uma classe de serviço que conversa com o Asaas.
4	**Cobrança na inscrição.** Integração no controller e tratamento de falhas.
5	**Webhooks.** O conceito e o primeiro aviso real, observado no webhook.site.
6	**Recebendo o aviso.** Endpoint de webhook com token e idempotência.
7	**Ponta a ponta.** Túnel, configuração final e o pagamento confirmando a inscrição.
8	**Síntese e exercícios.** Boas práticas de integração e prática.

Conferido com **PHP 8.5**, **Laravel 13.39**, API **v3** do sandbox do Asaas e **cloudflared 2026.10**.

ETAPA 1 · EXPLORAR ANTES DE PROGRAMAR

2 Conhecendo a API do Asaas

Integrar com um serviço externo começa pela **documentação**, e não pelo código. Antes de escrever uma linha de PHP, vamos fazer no Postman, à mão, exatamente as chamadas que a

nossa API fará depois. Assim, quando algo der errado no código, já saberemos como é a resposta certa.

DEFINIÇÃO — INTEGRAÇÃO DE SISTEMAS

Integrar dois sistemas é fazê-los trocar dados e acionar operações um no outro, cada um continuando independente. Na integração por API, um sistema faz o papel de **cliente** das operações que o outro expõe, seguindo um **contrato**: as URLs, os campos, os formatos e os códigos de resposta descritos na documentação. Nenhum dos lados conhece o código do outro; só o contrato.

QUADRO 2.1 — O CONTRATO DA API DO ASAAS QUE USAREMOS

Item	Valor
URL base (sandbox)	`https://api-sandbox.asaas.com/v3`
URL base (produção)	`https://api.asaas.com/v3` (não usaremos)
Autenticação	cabeçalho `access_token` com a chave da conta
Identificação	cabeçalho `User-Agent` com o nome da aplicação (obrigatório)
`POST /customers`	cria um cliente (pagador); obrigatórios: `name`, `cpfCnpj`
`POST /payments`	cria uma cobrança; obrigatórios: `customer`, `billingType`, `value`, `dueDate`
`POST /sandbox/payment/{id}/confirm`	(só no sandbox) simula o pagamento de uma cobrança

PASSO 1.1 — UM AMBIENTE DO POSTMAN PARA O ASAAS

Crie um ambiente novo, `Asaas sandbox`, com duas variáveis: `asaas_url` = `https://api-sandbox.asaas.com/v3` e `asaas_key` = a sua chave. Marque a `asaas_key` como tipo **secret**, para que ela não fique visível na tela. Crie a coleção `Asaas` e, nas configurações da coleção, aba **Authorization**, escolha **API Key** com *Key* `access_token`, *Value* `{{asaas_key}}` e *Add to* **Header**. Todas as requisições da coleção herdarão o cabeçalho. Em cada uma, acrescente também o `User-Agent: workshop-laravel`.

PASSO 1.2 — CRIAR UM CLIENTE

`POST` `{{asaas_url}}/customers` — `200 OK`

```
{
    "name": "Ana Souza",
    "cpfCnpj": "52998224725",
    "email": "ana@example.com",
```

```
    "notificationDisabled": true
}
```

RESPOSTA

```
{
    "object": "customer",
    "id": "cus_000009798129",
    "dateCreated": "2026-11-02",
    "name": "Ana Souza",
    "email": "ana@example.com",
    "cpfCnpj": "52998224725",
    "personType": "FISICA",
    "deleted": false,
    "notificationDisabled": true
}
```

Trecho da resposta: o objeto completo tem 30 campos, a maioria nula.

O `id` que começa com `cus_` identifica o cliente **dentro do Asaas.** Guarde-o: a cobrança será feita para ele. O campo `notificationDisabled` pede que o Asaas **não envie e-mails nem SMS** a esse cliente, o que é importante porque, mesmo no sandbox, as notificações podem ser enviadas de verdade. Repare também que o Asaas respondeu `200`, e não `201`, a uma criação. Cada API tem as suas convenções; quem integra se adapta ao contrato.

PASSO 1.3 — CRIAR UMA COBRANÇA

Use o `id` do cliente que você recebeu e uma data de vencimento **futura**, no formato `AAAA-MM-DD`:

POST {{asaas_url}}/payments 200 OK

```
{
    "customer": "cus_000009798129",
    "billingType": "UNDEFINED",
    "value": 25,
    "dueDate": "2026-11-05",
    "description": "Inscrição: Minicurso de Git e GitHub"
}
```

RESPOSTA


```
{
    "object": "payment",
    "id": "pay_qagnshapdh6ht0qy",
    "dateCreated": "2026-11-02",
    "customer": "cus_000009798129",
    "value": 25.0,
    "netValue": 24.01,
    "description": "Inscrição: Minicurso de Git e GitHub",
    "billingType": "UNDEFINED",
    "status": "PENDING",
    "dueDate": "2026-11-05",
    "invoiceUrl": "https://sandbox.asaas.com/i/qagnshapdh6ht0qy",
    "bankSlipUrl": "https://sandbox.asaas.com/b/pdf/qagnshapdh6ht0qy",
    "externalReference": null
}
```


Trecho da resposta: o objeto completo tem 38 campos.

O `billingType` `UNDEFINED` deixa o pagador escolher a forma de pagamento (Pix, boleto ou cartão); os outros valores possíveis são `PIX`, `BOLETO` e `CREDIT_CARD`. A cobrança nasce com status `PENDING`. Abra o `invoiceUrl` no navegador: é a **fatura** que o participante veria, com as opções de pagamento. O `netValue` é o valor líquido, já descontada a tarifa da plataforma.

PASSO 1.4 — OS ERROS DO ASAAS

Repita a criação sem o campo `dueDate`:

POST `{{asaas_url}}/payments` — 400 Bad Request

RESPOSTA


```
{
    "errors": [
        {
            "code": "invalid_dueDate",
            "description": "O parâmetro dueDate deve ser informado"
        }
    ]
}
```


E com uma chave inválida no ambiente, o erro é de autenticação:

GET `{{asaas_url}}/customers` — 401 Unauthorized

RESPOSTA


```
{
    "errors": [
        {
            "code": "invalid_access_token_format",
            "description": "O valor fornecido não parece ser uma chave de API válida do
Asaas. Verifique o formato da sua chave."
        }
    ]
}
```

Compare com os erros da nossa API: o Laravel devolve `422` e um objeto `errors` com os campos como chaves; o Asaas devolve `400` e uma **lista** de erros, cada um com `code` e `description`. Os dois formatos são válidos. A nossa integração precisará entender o formato *deles*.

PARA DISCUSSÃO EM SALA

1. Por que testar no Postman antes de escrever o código da integração? O que se ganha quando algo der errado depois?
2. O Asaas pede um `User-Agent` com o nome da aplicação. Que utilidade isso tem para quem opera a API?
3. O nosso sistema tem o `id` da inscrição; o Asaas tem o `id` da cobrança. Como cada lado vai encontrar o registro correspondente no outro?

ETAPA 2 · CREDENCIAIS

3 A chave no lugar certo

A chave de API não pode aparecer no código-fonte: o código vai para o Git, é compartilhado com a equipe e às vezes publicado. O lugar das credenciais é o arquivo `.env`, que fica só na máquina onde a aplicação roda e já vem listado no `.gitignore` do Laravel.

PASSO 2.1 — AS VARIÁVEIS NO .ENV

Acrescente ao final do `.env`, com a sua chave **entre aspas simples**:

.env (final do arquivo)

```
ASAAS_API_KEY='$aact_hmlg_000…cole-aqui-a-sua-chave…'
ASAAS_WEBHOOK_TOKEN=
```

A chave começa com `$`, e em vários contextos o cifrão tem significado especial (em PHP, dentro de aspas duplas, ele indica uma variável). As aspas simples deixam claro que o texto é literal. O `ASAAS_WEBHOOK_TOKEN` fica vazio por enquanto; ele será preenchido na Etapa 5. Para que quem clonar o projeto saiba que essas variáveis existem, acrescente as mesmas linhas, **sem os valores**, ao arquivo `.env.example`, que esse sim vai para o Git.

PASSO 2.2 — O SERVIÇO EM CONFIG/SERVICES.PHP

O Laravel recomenda não ler o `.env` espalhado pelo código, e sim reuni-lo nos arquivos de `config/`. O `services.php` existe justamente para credenciais de serviços de terceiros. Acrescente, logo depois do `return [`:

config/services.php (acréscimo)

```
    'asaas' => [
        'url' => env('ASAAS_URL', 'https://api-sandbox.asaas.com/v3'),
        'key' => env('ASAAS_API_KEY'),
        'webhook_token' => env('ASAAS_WEBHOOK_TOKEN'),
    ],
```

Daqui em diante, o código pede `config('services.asaas.key')`. A URL tem um valor padrão, o sandbox: para ir para produção um dia, basta definir `ASAAS_URL` e trocar a chave no `.env` do servidor, sem tocar no código.

SE A CHAVE VAZAR

Chave publicada por engano no GitHub, num print ou num grupo deve ser considerada **comprometida**, mesmo que o commit seja apagado depois: há robôs que varrem repositórios públicos à procura de credenciais. O procedimento é sempre o mesmo: gerar uma chave nova no painel, trocar no `.env` e desativar a antiga.

4

ETAPA 3 · CLIENTE HTTP

Uma classe que conversa com o Asaas

Até aqui, a nossa API só *respondia* requisições. Agora ela também vai *fazer* requisições, como o Postman fez na Etapa 1. O Laravel tem um **cliente HTTP** embutido, a fachada `Http`, e vamos concentrar tudo o que diz respeito ao Asaas numa única classe.

DEFINIÇÃO — CLASSE DE SERVIÇO

Uma **classe de serviço** isola uma responsabilidade que não pertence ao controller nem ao model, como falar com um sistema externo. O controller pede *"crie uma cobrança"*, sem saber de URLs, cabeçalhos ou formatos. Se o Asaas mudar algo, ou se um dia a plataforma for outra, só essa classe muda.

PASSO 3.1 — A CLASSE ASAAS

Crie a pasta `app/Services` e, nela, o arquivo:

app/Services/Asaas.php

```php
<?php

namespace App\Services;

use Illuminate\Http\Client\PendingRequest;
use Illuminate\Support\Facades\Http;

class Asaas
{
    private function http(): PendingRequest
    {
        return Http::baseUrl(config('services.asaas.url'))
            ->withHeaders([
                'access_token' => config('services.asaas.key'),
                'User-Agent' => 'inscricoes-workshop',
            ])
            ->acceptJson()
            ->timeout(15)
            ->throw();
    }

    public function criarCliente(string $nome, string $email, string $cpf): array
    {
        return $this->http()->post('/customers', [
            'name' => $nome,
            'email' => $email,
            'cpfCnpj' => $cpf,
            'notificationDisabled' => true,
        ])->json();
    }

    public function criarCobranca(
        string $clienteId,
        float $valor,
        string $descricao,
        string $referencia,
    ): array {
        return $this->http()->post('/payments', [
            'customer' => $clienteId,
            'billingType' => 'UNDEFINED',
            'value' => $valor,
            'dueDate' => now()->addDays(3)->toDateString(),
            'description' => $descricao,
            'externalReference' => $referencia,
        ])->json();
    }
}
```

O método privado `http()` prepara uma requisição com tudo o que é comum a todas as chamadas, na mesma ordem em que configuramos o Postman:

- `baseUrl`: as chamadas usam só o caminho (`/customers`), e o Laravel junta com a URL base;
- `withHeaders`: a chave no `access_token` e o `User-Agent`;

- `acceptJson`: pede respostas em JSON; o `post()` já envia o corpo como JSON;
- `timeout(15)`: desiste depois de 15 segundos, em vez de deixar o participante esperando para sempre se o Asaas não responder;
- `throw()`: se a resposta for `4xx` ou `5xx`, lança uma exceção (`RequestException`) em vez de devolver o erro silenciosamente.

Os dois métodos públicos montam o corpo igual ao do Postman e devolvem a resposta já convertida em array, com `->json()`. Na cobrança, o vencimento é calculado (daqui a três dias) e o `externalReference` leva o `id` da nossa inscrição: é a resposta à terceira pergunta da discussão anterior. O Asaas devolverá esse valor em tudo o que disser sobre a cobrança.

POR QUE A CLASSE NÃO TEM CONSTRUTOR NEM É REGISTRADA EM LUGAR NENHUM?

O Laravel tem um **contêiner de serviços** que cria objetos automaticamente. Basta declarar `Asaas $asaas` como parâmetro de um método de controller, e o framework instancia a classe e a entrega pronta. É a mesma mágica que entregou o `Request $request` nos controllers do encontro anterior.

5

ETAPA 4 · INTEGRAÇÃO DE SAÍDA

Cobrança na inscrição

A inscrição precisa guardar a ligação com o Asaas: o cliente e a cobrança criados lá, o link de pagamento para devolver ao participante e, mais tarde, a data do pagamento.

PASSO 4.1 — NOVAS COLUNAS

```
PS> php artisan make:migration add_asaas_to_inscricoes_table
```

TERMINAL

```
   INFO  Migration [database/migrations/2026_11_02_174125_add_asaas_to_inscricoes_table.php]
created successfully.
```

database/migrations/…_add_asaas_to_inscricoes_table.php (trecho)

```php
    public function up(): void
    {
        Schema::table('inscricoes', function (Blueprint $table) {
            $table->string('asaas_cliente_id')->nullable();
            $table->string('asaas_cobranca_id')->nullable()->unique();
            $table->string('link_pagamento')->nullable();
            $table->timestamp('pago_em')->nullable();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('inscricoes', function (Blueprint $table) {
            $table->dropColumn([
                'asaas_cliente_id', 'asaas_cobranca_id', 'link_pagamento', 'pago_em',
            ]);
        });
    }
```

Uma migration que **altera** uma tabela usa `Schema::table` em vez de `Schema::create`, e o método `down()` desfaz a alteração. O `unique()` na cobrança vai acelerar e garantir a busca que o webhook fará. Rode `php artisan migrate`. Depois, no model, libere os novos campos e converta a data de pagamento:

app/Models/Inscricao.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Inscricao extends Model
{
    protected $table = 'inscricoes';

    protected $fillable = [
        'user_id', 'nome', 'email', 'cpf', 'status',
        'asaas_cliente_id', 'asaas_cobranca_id', 'link_pagamento', 'pago_em',
    ];

    protected function casts(): array
    {
        return ['pago_em' => 'datetime:Y-m-d H:i:s'];
    }

    public function evento(): BelongsTo
    {
        return $this->belongsTo(Evento::class);
    }
}
```

PASSO 4.2 — O STORE, AGORA COM COBRANÇA

No `InscricaoController`, acrescente os três `use` e o parâmetro `Asaas $asaas` no `store`. O bloco novo entra depois do `create`:

app/Http/Controllers/InscricaoController.php (trechos novos)

```php
use App\Services\Asaas;
use Illuminate\Http\Client\ConnectionException;
use Illuminate\Http\Client\RequestException;

    public function store(Request $request, Evento $evento, Asaas $asaas)
    {
        // ... validação, vagas, dono e status: como no encontro anterior ...

        $inscricao = $evento->inscricoes()->create($dados);

        if ($evento->valor > 0) {
            try {
                $cliente = $asaas->criarCliente(
                    $inscricao->nome,
                    $inscricao->email,
                    $inscricao->cpf,
                );
                $cobranca = $asaas->criarCobranca(
                    $cliente['id'],
                    $evento->valor,
                    "Inscrição: {$evento->titulo}",
                    (string) $inscricao->id,
                );
            } catch (RequestException|ConnectionException $e) {
                report($e);
                $inscricao->delete();

                return response()->json([
                    'message' => 'Não foi possível gerar a cobrança. Tente novamente.',
                ], 502);
            }

            $inscricao->update([
                'asaas_cliente_id' => $cliente['id'],
                'asaas_cobranca_id' => $cobranca['id'],
                'link_pagamento' => $cobranca['invoiceUrl'],
            ]);
        }

        return response()->json($inscricao, 201);
    }
```

Para eventos gratuitos, nada muda. Para eventos pagos, o controller pede ao serviço o cliente e a cobrança e grava na inscrição os identificadores e o `invoiceUrl`. O ponto delicado é o `catch`. A inscrição já foi gravada quando o Asaas é chamado; se a chamada falhar, por dados recusados (`RequestException`) ou por falta de conexão (`ConnectionException`), ficaria no banco uma inscrição pendente **sem cobrança**, ocupando vaga e impossível de pagar. Por isso o `catch` registra o erro no log (`report`), **desfaz a inscrição** e responde `502 Bad Gateway`: o código que diz “um serviço do qual eu dependo falhou”.

PASSO 4.3 — INSCRIÇÃO PAGA DE VERDADE

Recrie o banco para começar limpo (`php artisan migrate:fresh --seed`). O comando também apaga os usuários e os tokens: refaça o cadastro no Postman (*Auth — cadastrar*), que grava um token novo, e inscreva alguém no evento 2:

POST {{base_url}}/eventos/2/inscricoes — **201 Created**

```
{ "nome": "Bruno Lima", "email": "bruno@example.com", "cpf": "11144477735" }
```

RESPOSTA


```
{
    "nome": "Bruno Lima",
    "email": "bruno@example.com",
    "cpf": "11144477735",
    "user_id": 1,
    "status": "pendente",
    "evento_id": 2,
    "updated_at": "2026-11-02T20:38:32.000000Z",
    "created_at": "2026-11-02T20:38:30.000000Z",
    "id": 1,
    "asaas_cliente_id": "cus_000009798134",
    "asaas_cobranca_id": "pay_d2d6a2ry0eozd37g",
    "link_pagamento": "https://sandbox.asaas.com/i/d2d6a2ry0eozd37g"
}
```


A inscrição é de quem estava logado (`"user_id": 1`, a conta recém-cadastrada), como no encontro anterior. Em pouco mais de um segundo, a nossa API fez duas requisições ao Asaas e devolveu ao participante o link de pagamento. No painel do sandbox, em **Cobranças**, a cobrança de R$ 25,00 aparece para Bruno Lima. Abra o `link_pagamento` no navegador: é a fatura real do ambiente de testes.

PASSO 4.4 — QUANDO O ASAAS RECUSA

O CPF `12345678900` tem 11 dígitos e passa na nossa validação, mas os dígitos verificadores estão errados:

POST {{base_url}}/eventos/2/inscricoes — **502 Bad Gateway**

```
{ "nome": "Carla Dias", "email": "carla@example.com", "cpf": "12345678900" }
```

RESPOSTA


```
{
    "message": "Não foi possível gerar a cobrança. Tente novamente."
}
```


O participante recebe uma mensagem clara; o desenvolvedor encontra o motivo técnico no log, `storage/logs/laravel.log`:

```
TERMINAL
[2026-11-02 17:39:10] local.ERROR: HTTP request returned status code 400:
{"errors":[{"code":"invalid_object","description":"O CPF/CNPJ informado é inválido."}]}
 {"exception":"[object] (Illuminate\\Http\\Client\\RequestException(code: 400): …
```

E a inscrição de Carla não ficou no banco: confira na tabela `inscricoes`, abrindo o `database.sqlite` no VS Code. Pela API, `GET {{base_url}}/eventos/2/inscricoes` responde `403`: o evento 2 veio do seeder, não tem dono, e a lista de inscritos é só do dono.

CONFIRA ANTES DE SEGUIR

A inscrição de Bruno tem `asaas_cobranca_id` e `link_pagamento` preenchidos, e a cobrança aparece no painel do sandbox. Se a resposta for `502` para todo mundo, abra o log: um `401` do Asaas indica chave errada no `.env` (depois de alterar o `.env`, reinicie o `php artisan serve`).

PARA DISCUSSÃO EM SALA

1. A cobrança foi criada no Asaas, mas a gravação final no nosso banco falhou. Que inconsistência sobra? Como seria possível detectá-la depois?
2. Por que `502`, e não `422` ou `500`?
3. A requisição de inscrição agora demora o tempo de duas chamadas externas. Em que situações isso seria um problema?

6

ETAPA 5 · INTEGRAÇÃO DE ENTRADA

Webhooks: quando o outro sistema avisa

Bruno vai pagar. Quando? Pode ser em um minuto, pode ser amanhã, pode ser nunca. Como a nossa API fica sabendo? Há dois jeitos, e a Figura 6.1 compara os dois.

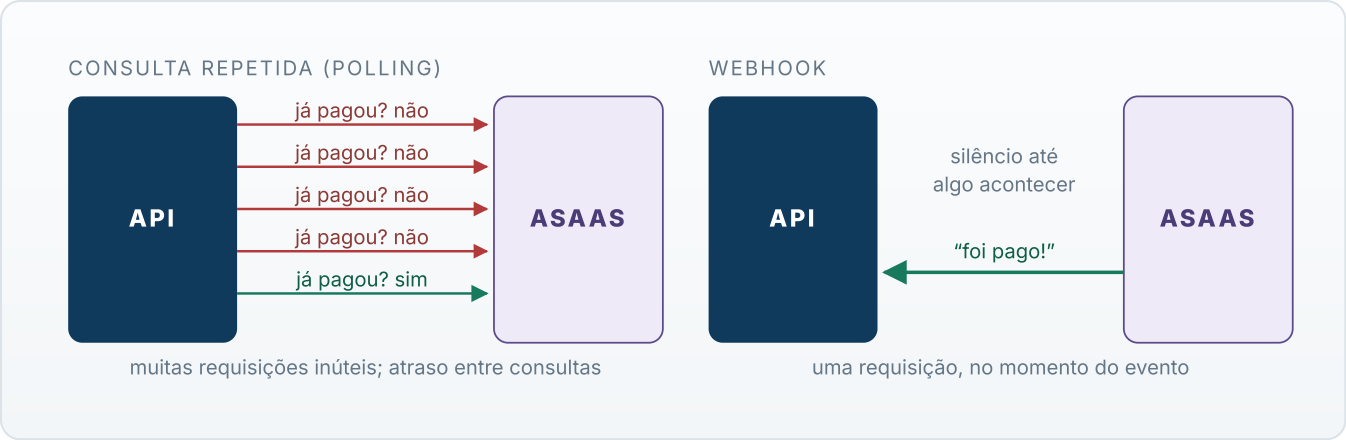


Figura 6.1 — Consultar repetidamente (à esquerda) e ser avisado (à direita).

DEFINIÇÃO — WEBHOOK

Um **webhook** é uma requisição HTTP que um sistema **envia por iniciativa própria** a uma URL cadastrada por outro sistema, quando um **evento** acontece. É uma inversão de papéis: o serviço externo vira o cliente, e a nossa API, o servidor que recebe. Para isso, a nossa API precisa de uma rota para o aviso, e essa rota precisa ser alcançável pela internet.

O Asaas envia webhooks para dezenas de eventos. O Quadro 6.1 traz os de cobrança que nos interessam. No sandbox, confirmar uma cobrança Pix ou boleto gera o `PAYMENT_RECEIVED`; um pagamento por cartão passa antes por `PAYMENT_CONFIRMED`. Por isso trataremos os dois como "pago".

QUADRO 6.1 — EVENTOS DE COBRANÇA DO ASAAS USADOS NO WORKSHOP

Evento	Quando acontece	Efeito na inscrição
`PAYMENT_CREATED`	Uma cobrança foi criada	nenhum
`PAYMENT_CONFIRMED`	Pagamento confirmado (ex.: cartão aprovado), dinheiro ainda não disponível	confirmada
`PAYMENT_RECEIVED`	Pagamento recebido (Pix, boleto compensado)	confirmada
`PAYMENT_OVERDUE`	Venceu sem pagamento	(exercício)
`PAYMENT_DELETED` / `PAYMENT_REFUNDED`	Cobrança removida / pagamento estornado	cancelada

Observando um webhook real

Antes de escrever o código que recebe o aviso, vamos **ver** um. O site `https://webhook.site` gera, ao ser aberto, uma URL única que aceita qualquer requisição e mostra na tela tudo o que

chegou: método, cabeçalhos e corpo. É o "Postman ao contrário".

PASSO 5.1 — CADASTRAR O WEBHOOK NO ASAAS

1. Abra `https://webhook.site` e copie a URL exclusiva que aparece (`https://webhook.site/…`).
2. No painel do sandbox, abra o menu do usuário e vá em **Integrações → Webhooks → Criar Webhook**.
3. Preencha: **nome** `Workshop`; **URL**, a do webhook.site; **e-mail**, o seu (o Asaas avisa por ele se o webhook falhar); **versão da API** `v3`; envio **sequencial**; webhook **ativo**.
4. **Token de autenticação:** use o botão de gerar token (ou digite um texto aleatório de 32 a 255 caracteres, sem espaços). **Copie o token** e cole-o no `.env`, na linha `ASAAS_WEBHOOK_TOKEN=`. Ele não será exibido de novo.
5. Em **eventos**, marque os de cobrança do Quadro 6.1 e salve.

PASSO 5.2 — SIMULAR O PAGAMENTO

No sandbox ninguém paga de verdade: o pagamento é **simulado**. Isso pode ser feito pelo painel, na tela da cobrança (opção de confirmar o pagamento, disponível para Pix e boleto), ou pela API, com um endpoint que só existe no sandbox. Use o `id` da cobrança criada no Passo 1.3:

```
POST  {{asaas_url}}/sandbox/payment/pay_qagnshapdh6ht0qy/confirm  200 OK
```

RESPOSTA


```
{
    "object": "payment",
    "id": "pay_qagnshapdh6ht0qy",
    "customer": "cus_000009798129",
    "value": 25.0,
    "netValue": 24.01,
    "billingType": "BOLETO",
    "status": "RECEIVED",
    "dueDate": "2026-11-05",
    "paymentDate": "2026-11-02",
    "invoiceUrl": "https://sandbox.asaas.com/i/qagnshapdh6ht0qy"
}
```


Trecho da resposta. A cobrança `UNDEFINED` foi confirmada como boleto.

Em poucos segundos, o webhook.site mostra uma requisição `POST` do Asaas com o evento `PAYMENT_RECEIVED`. (Daqui em diante, cada cobrança criada gerará também um `PAYMENT_CREATED`.) Abaixo, o corpo de um `PAYMENT_RECEIVED` real, recebido pelo webhook.site quando foi confirmada a cobrança da inscrição de Bruno, criada pela nossa API no Passo 4.3:

RESPOSTA


```
{
    "id": "evt_d26e303b238e509335ac9ba210e51b0f&23996372",
    "event": "PAYMENT_RECEIVED",
    "dateCreated": "2026-11-02 17:38:43",
    "account": {
        "id": "6fa6af1d-…",
        "ownerId": null
    },
    "payment": {
        "object": "payment",
        "id": "pay_d2d6a2ry0eozd37g",
        "customer": "cus_000009798134",
        "value": 25.0,
        "billingType": "BOLETO",
        "status": "RECEIVED",
        "paymentDate": "2026-11-02",
        "invoiceUrl": "https://sandbox.asaas.com/i/d2d6a2ry0eozd37g",
        "externalReference": "1"
    }
}
```


Trecho do corpo: o objeto `payment` completo tem 40 campos, os mesmos de uma consulta à cobrança.

Três detalhes do que chegou orientam todo o código da próxima etapa:

- Entre os cabeçalhos está `asaas-access-token`, com o token que você cadastrou. É a forma de a nossa API saber que o aviso veio mesmo do Asaas: a URL do webhook não é secreta, e qualquer um poderia enviar um JSON dizendo "foi pago". O `User-Agent` é `Asaas_Hmlg/3.0`.
- O `id` `evt_…` identifica **o aviso**. A documentação do Asaas avisa que a entrega é "pelo menos uma vez": o mesmo evento pode chegar repetido, e os reenvios mantêm o mesmo `id`.
- O `payment.id` é a cobrança, e o `externalReference` é o `id` da nossa inscrição, que enviamos ao criar a cobrança.

Deixe a aba do webhook.site aberta: na próxima etapa vamos copiar dela um corpo real.

7

ETAPA 6 · O ENDPOINT DE WEBHOOK

Recebendo o aviso com segurança

O endpoint que recebe webhooks é uma rota como outra qualquer, mas exposta a um mundo que não controlamos. Ele precisa responder a três perguntas, nesta ordem: **quem está chamando?** (autenticidade), **já vi este aviso?** (idempotência) e **o que muda no meu sistema?** (efeito).

DEFINIÇÃO — IDEMPOTÊNCIA

Uma operação é **idempotente** quando executá-la várias vezes produz o mesmo resultado que executá-la uma vez. Receber duas vezes o aviso "foi pago" não pode, por exemplo, enviar dois e-mails de confirmação nem registrar o pagamento duas vezes. A técnica recomendada pelo Asaas é guardar o `id` de cada evento processado numa coluna `unique` e ignorar os que já estiverem lá.

PASSO 6.1 — A TABELA DE NOTIFICAÇÕES

```
PS> php artisan make:model Notificacao
PS> php artisan make:migration create_notificacoes_table
```

database/migrations/…_create_notificacoes_table.php (trecho)

```php
        Schema::create('notificacoes', function (Blueprint $table) {
            $table->id();
            $table->string('asaas_evento_id')->unique();
            $table->string('tipo');
            $table->json('payload');
            $table->timestamps();
        });
```

app/Models/Notificacao.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Notificacao extends Model
{
    protected $table = 'notificacoes';

    protected $fillable = ['asaas_evento_id', 'tipo', 'payload'];

    protected function casts(): array
    {
        return ['payload' => 'array'];
    }
}
```

Mais uma vez o plural em português exige o `$table`. A coluna `payload` guarda o corpo inteiro do aviso: além de servir à idempotência, a tabela vira um **histórico** de tudo o que o Asaas disse à nossa API, precioso para investigar problemas. Rode `php artisan migrate`.

PASSO 6.2 — O CONTROLLER DO WEBHOOK

```
PS> php artisan make:controller AsaasWebhookController --invokable
```

```
TERMINAL
   INFO  Controller [app/Http/Controllers/AsaasWebhookController.php] created successfully.
```

app/Http/Controllers/AsaasWebhookController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Inscricao;
use App\Models\Notificacao;
use Illuminate\Http\Request;

class AsaasWebhookController extends Controller
{
    public function __invoke(Request $request)
    {
        // 1. Quem está chamando? Só o Asaas conhece o token.
        $token = config('services.asaas.webhook_token');
        $enviado = (string) $request->header('asaas-access-token');

        if (! $token || ! hash_equals($token, $enviado)) {
            return response()->json(['message' => 'Token inválido.'], 401);
        }

        // 2. Já processamos este evento? O Asaas pode enviar o mesmo mais de uma vez.
        $notificacao = Notificacao::firstOrCreate(
            ['asaas_evento_id' => $request->input('id')],
            ['tipo' => $request->input('event'), 'payload' => $request->all()],
        );

        if (! $notificacao->wasRecentlyCreated) {
            return response()->json(['recebido' => true, 'duplicado' => true]);
        }

        // 3. A qual inscrição a cobrança pertence?
        $cobrancaId = $request->input('payment.id');
        $inscricao = Inscricao::where('asaas_cobranca_id', $cobrancaId)->first();

        if ($inscricao) {
            $tipo = $request->input('event');
            $pago = in_array($tipo, ['PAYMENT_CONFIRMED', 'PAYMENT_RECEIVED']);
            $cancelado = in_array($tipo, ['PAYMENT_DELETED', 'PAYMENT_REFUNDED']);

            if ($pago && $inscricao->status !== 'confirmada') {
                $inscricao->update(['status' => 'confirmada', 'pago_em' => now()]);
            }

            if ($cancelado) {
                $inscricao->update(['status' => 'cancelada']);
            }
        }

        return response()->json(['recebido' => true]);
    }
}
```

Um controller `--invokable` tem uma única ação, o método `__invoke`, apropriado para um endpoint que faz uma coisa só. Os três blocos respondem às três perguntas:

1. **Autenticidade.** O token do cabeçalho é comparado com o do `.env` usando `hash_equals`, que leva o mesmo tempo para acertar ou errar e, assim, não dá pistas a quem tenta adivinhar o token. O `! $token` fecha uma porta sutil: sem ele, se a variável estivesse vazia, uma requisição *sem* o cabeçalho seria aceita, porque vazio é igual a vazio.
2. **Idempotência.** `firstOrCreate` procura a notificação pelo `id` do evento e só cria se não existir. A propriedade `wasRecentlyCreated` diz se ela é nova; se não for, o aviso é repetido, e a resposta é `200` sem fazer mais nada. Responder erro a um repetido faria o Asaas tentar de novo.
3. **Efeito.** A inscrição é localizada pela cobrança. Pagamento confirmado ou recebido confirma a inscrição, uma única vez, mesmo que cheguem os dois eventos; remoção ou estorno a cancela. Eventos desconhecidos são registrados e ignorados, sem erro.

Falta a rota. Ao final de `routes/api.php`, **fora** do grupo `auth:sanctum`, com o `use` correspondente no topo. Quem chama o webhook é o Asaas, que não tem um token do Sanctum: a proteção desta rota é o token próprio do webhook, conferido pelo controller.

```php
use App\Http\Controllers\AsaasWebhookController;

Route::post('/webhooks/asaas', AsaasWebhookController::class);
```

RESPONDA 200, E RÁPIDO

Para o Asaas, só uma resposta de sucesso conta como entrega; a documentação recomenda responder exatamente `200`. Qualquer outra coisa, inclusive demorar demais, é falha, e o aviso volta a ser enviado em intervalos crescentes. Depois de **15 falhas seguidas**, a fila daquele webhook é **interrompida** e os avisos seguintes ficam retidos até alguém reativá-la no painel. Por isso o endpoint faz pouco e responde logo. Em sistemas maiores, o processamento pesado vai para uma *fila* (*job*) e o webhook só registra e responde.

PASSO 6.3 — TESTAR COM UM AVISO REAL, PELO POSTMAN

A nossa API ainda não é alcançável pela internet, mas podemos fazer o papel do Asaas. Confirme, com o endpoint do Passo 5.2, a cobrança da inscrição de Bruno (o `asaas_cobranca_id` do Passo 4.3). No webhook.site, abra o `PAYMENT_RECEIVED` dessa cobrança e copie o corpo inteiro. No Postman, cole-o como **Body → raw → JSON:**

POST `{{base_url}}/webhooks/asaas` **401 Unauthorized**

RESPOSTA

```
{
    "message": "Token inválido."
}
```

Sem o cabeçalho, a porta está fechada. Acrescente `asaas-access-token` com o token do seu `.env` e envie de novo:

POST `{{base_url}}/webhooks/asaas` **200 OK**

RESPOSTA

```
{
    "recebido": true
}
```

Envie **exatamente o mesmo corpo** mais uma vez, como faria um reenvio do Asaas:

POST `{{base_url}}/webhooks/asaas` **200 OK**

RESPOSTA

```
{
    "recebido": true,
    "duplicado": true
}
```

PASSO 6.4 — A INSCRIÇÃO CONFIRMADA

GET {{base_url}}/inscricoes/1 200 OK

RESPOSTA


```
{
    "id": 1,
    "evento_id": 2,
    "nome": "Bruno Lima",
    "email": "bruno@example.com",
    "cpf": "11144477735",
    "status": "confirmada",
    "created_at": "2026-11-02T20:38:30.000000Z",
    "updated_at": "2026-11-02T20:39:01.000000Z",
    "asaas_cliente_id": "cus_000009798134",
    "asaas_cobranca_id": "pay_d2d6a2ry0eozd37g",
    "link_pagamento": "https://sandbox.asaas.com/i/d2d6a2ry0eozd37g",
    "pago_em": "2026-11-02 17:39:01",
    "user_id": 1,
    "evento": {
        "id": 2,
        "titulo": "Minicurso de Git e GitHub",
        "valor": "25.00",
        …
    }
}
```


Trecho: os demais campos do evento foram omitidos.

O status é `confirmada` e o `pago_em` está preenchido. A consulta funciona porque a inscrição é da conta logada; com outra conta, a resposta seria `403`. Abra o `database.sqlite` no VS Code: a tabela `notificacoes` tem uma linha por aviso recebido, sem a duplicata.

PARA DISCUSSÃO EM SALA

1. Por que a idempotência é verificada **depois** do token, e não antes?
2. Um aviso chega para uma cobrança que não existe no nosso banco. A resposta `200` está certa? O que mais deveria acontecer?
3. Os avisos podem chegar fora de ordem? O que aconteceria se um `PAYMENT_REFUNDED` chegasse antes do `PAYMENT_RECEIVED`?

8

ETAPA 7 · PONTA A PONTA

Túnel, configuração final e o ciclo completo

Falta tirar o Postman do meio: o Asaas precisa entregar o aviso diretamente à nossa API. Ela roda em `127.0.0.1:8000`, um endereço que só existe dentro do computador. O **cloudflared**, instalado no material prévio, cria um endereço público temporário que encaminha tudo para lá.

PASSO 7.1 — ABRIR O TÚNEL

Com o `php artisan serve` rodando, abra **outro** terminal:

```
PS> cloudflared tunnel --url http://127.0.0.1:8000
```

O cloudflared escreve várias linhas de informação e, entre elas, um quadro com a URL pública, no formato `https://quatro-palavras-aleatorias.trycloudflare.com`. Copie essa URL. Teste-a no navegador, acrescentando `/api/ping` ao final: deve aparecer o JSON do primeiro encontro, agora vindo da internet. O túnel dura enquanto o comando estiver rodando, e a URL **muda a cada execução.**

> **SUA API ESTÁ NA INTERNET**
>
> Enquanto o túnel estiver aberto, **qualquer pessoa** que conheça a URL alcança a sua API. As rotas de eventos e inscrições pedem login, mas o cadastro é aberto: qualquer um cria uma conta e, com ela, vê os eventos, cria eventos novos e se inscreve. Apagar ou alterar os seus eventos ele não consegue, graças às regras de dono, mas isso não impede o abuso. Use o túnel só durante o teste, com dados fictícios, e encerre-o com `Ctrl` + `C` ao terminar. É também por isso que o webhook exige o token: a URL não é segredo.

PASSO 7.2 — APONTAR O WEBHOOK PARA A API

No painel do sandbox, em **Integrações → Webhooks**, edite o webhook `Workshop` e troque a URL do webhook.site por `https://…trycloudflare.com/api/webhooks/asaas`. Mantenha o mesmo token, que é o que está no seu `.env`. Se o painel indicar que a fila está **interrompida** (por falhas anteriores), reative-a.

PASSO 7.3 — O CICLO COMPLETO

1. Crie uma inscrição nova no evento pago, que ainda tem uma vaga. Se precisar de mais, crie com *Eventos — criar* um evento pago seu (`"valor": 25`) e use-o: o evento 2 veio do seeder, não tem dono, e um `PATCH` nele responde `403`. A resposta traz `status: pendente` e o `link_pagamento`.
2. Confirme o pagamento, pela tela da cobrança no painel do sandbox ou pelo `POST {{asaas_url}}/sandbox/payment/{id}/confirm`.
3. Observe o terminal do `php artisan serve`: aparece uma requisição `POST /api/webhooks/asaas`. Ninguém a enviou do Postman: foi o Asaas.
4. Consulte a inscrição, com a mesma conta que a criou: `confirmada`, com `pago_em`.
5. No painel, em **Integrações → Logs de Webhooks**, veja o envio com a resposta `200` da sua API.

CONFIRA: O CICLO FECHOU

Uma requisição criou a inscrição e a cobrança; o pagamento aconteceu **fora** da nossa API; e, sem nenhuma ação manual, a inscrição mudou de estado. É o fluxo da Figura 1.1 funcionando com uma plataforma de pagamentos real, no ambiente de testes. Em produção, o que muda é a chave, a URL base e o endereço do webhook, que passa a ser o do servidor da aplicação no lugar do túnel.

Testes automatizados da integração

Testar à mão, com túnel e painel, é ótimo para entender, mas lento para repetir. O Laravel permite testar a integração **sem chamar o Asaas**: `Http::fake()` intercepta as requisições de saída e devolve respostas combinadas, e `postJson` faz requisições à própria API. O projeto de referência do workshop traz o arquivo `tests/Feature/InscricaoAsaasTest.php`, com quatro testes: a cobrança gerada na inscrição, a inscrição desfeita quando o Asaas recusa, a confirmação idempotente pelo webhook e a recusa quando o token não está configurado. Como as rotas exigem login, o teste entra como um usuário com `Sanctum::actingAs()`, que cria o evento pago por `$user->eventos()` e, portanto, é o dono dele. Dois outros arquivos cobrem o encontro anterior: `tests/Feature/AuthTest.php` (cadastro, senha errada, `401` com e sem `Accept`, logout que invalida o token) e `tests/Feature/EventoDonoTest.php`, que alterna entre duas contas com `Sanctum::actingAs()` e confere cada regra de dono, inclusive os `403`. Para rodar todos, use `php artisan test`, sem o filtro.

```
PS> php artisan test --filter=InscricaoAsaasTest
```

```
TERMINAL
   PASS  Tests\Feature\InscricaoAsaasTest
  √ inscricao em evento pago gera cobranca                              0.46s
  √ falha no asaas desfaz inscricao                                     0.08s
  √ webhook confirma e e idempotente                                    0.04s
  √ webhook sem token configurado recusa                                0.02s

  Tests:    4 passed (12 assertions)
  Duration: 0.82s
```

9

SÍNTESE TÉCNICA

Integrar é cuidar das bordas

Integrar não é só fazer a chamada funcionar quando tudo dá certo: é decidir o que acontece em cada falha possível na fronteira entre os dois sistemas. O Quadro 9.1 reúne as decisões que tomamos.

QUADRO 9.1 — DECISÕES DE INTEGRAÇÃO DO WORKSHOP

Preocupação	O que fizemos	Onde
Credenciais	Chave e token só no `.env`, lidos por `config/services.php`	.env, config
Acoplamento	Tudo sobre o Asaas numa classe de serviço	App\Services\Asaas
Serviço lento ou fora	`timeout(15)` e captura de `ConnectionException`	Asaas, controller
Erro do serviço	`throw()`, log com `report()`, inscrição desfeita, `502`	InscricaoController
Correlação	`externalReference` = id da inscrição; `asaas_cobranca_id` na inscrição	migration, serviço
Aviso falso	Token no cabeçalho, comparado com `hash_equals`	AsaasWebhookController
Aviso repetido	`id` do evento em coluna `unique`; repetido responde 200	notificacoes
Rastreabilidade	Corpo de cada aviso guardado em `payload`	notificacoes

QUADRO 9.2 — ERROS FREQUENTES NESTE ENCONTRO

Sintoma	Causa e solução
A nossa API responde `401 Unauthenticated`	Token do login ausente ou apagado (por logout ou `migrate:fresh`). Faça login, ou cadastre-se de novo, no Postman.
A nossa API responde `403 This action is unauthorized.`	A conta logada não é a dona: o evento veio do seeder (sem dono) ou a inscrição foi feita por outra conta. Use um evento e uma inscrição criados pela conta do token.
Toda inscrição paga dá `502`	Veja o log. `401` do Asaas: chave errada, sem aspas simples ou `.env` alterado sem reiniciar o `artisan serve`.
Asaas responde `400` com `invalid_object`	CPF com dígitos verificadores inválidos. Use os CPFs fictícios do material.
Webhook responde `401`	Token do painel diferente do `ASAAS_WEBHOOK_TOKEN`, ou variável vazia.
Aviso não chega à API	URL do túnel mudou (nova execução do cloudflared), falta `/api/webhooks/asaas` no fim, ou a fila está interrompida no painel.
Aviso chega, mas a inscrição não muda	Cobrança criada antes da migration nova, sem `asaas_cobranca_id` gravado; ou evento não tratado (veja a tabela `notificacoes`).
Pix desabilitado no sandbox	Cadastro da conta incompleto. Revise os dados comerciais no painel.

10

PRÁTICA

Exercícios

ORIENTAÇÕES

Para cada exercício, consulte a documentação do Asaas (`https://docs.asaas.com`), teste a chamada no Postman **antes** de programar e só então implemente. Use sempre dados fictícios. As rotas novas da nossa API entram **dentro** do grupo `auth:sanctum`, e as que expõem uma inscrição seguem a regra do `show`: só quem a fez, com `Gate::authorize('view', $inscricao)`.

1 Pagar com Pix

Crie `GET /api/inscricoes/{inscricao}/pix`, que devolve o código "copia e cola" e a data de expiração do QR Code Pix da cobrança da inscrição.

Dica: `GET /v3/payments/{id}/pixQrCode` devolve `payload` (o copia e cola), `encodedImage` (a imagem em Base64) e `expirationDate`. Acrescente um método `qrCodePix()` à classe de serviço.

2 Cobrança vencida

Trate o evento `PAYMENT_OVERDUE`, mudando a inscrição para `vencida`. Teste no sandbox com `POST /v3/sandbox/payment/{id}/overdue`, que força o vencimento.

Dica: o evento precisa estar marcado na configuração do webhook. E uma inscrição vencida deve continuar ocupando vaga?

3 Cancelar também no Asaas

Ao cancelar uma inscrição pendente (exercício 3 do encontro anterior), remova a cobrança no Asaas, para que o link de pagamento deixe de funcionar.

Dica: `DELETE /v3/payments/{id}`. A remoção dispara o `PAYMENT_DELETED`, que o webhook já trata. O cancelamento local deve esperar o webhook ou acontecer na hora?

4 Não duplicar clientes

A mesma pessoa inscrita em dois eventos pagos gera hoje dois clientes no Asaas. Antes de criar, procure um cliente com o mesmo CPF e reutilize-o.

Dica: `GET /v3/customers?cpfCnpj=…` devolve uma lista; o cliente estará em `data[0]`, se existir. Implemente em um método `clientePorCpf()` do serviço.

5 Histórico de notificações

Crie `GET /api/inscricoes/{inscricao}/notificacoes`, que lista, em ordem, o tipo e a data de cada aviso do Asaas referente à cobrança da inscrição.

Dica: o id da cobrança está dentro do `payload`. Com o cast `array`, é possível filtrar com `where('payload->payment->id', …)`.

6 Mais um teste automatizado

Escreva um teste que garanta que um `PAYMENT_REFUNDED` recebido pelo webhook muda a inscrição para `cancelada`, seguindo o modelo de `InscricaoAsaasTest`.

Dica: não é preciso `Http::fake`: o webhook não chama o Asaas. Basta `postJson` com o cabeçalho do token.

Desafio E se o webhook nunca chegar?

Webhooks falham: túnel fechado, servidor fora do ar, fila interrompida. Crie `POST /api/inscricoes/{inscricao}/sincronizar`, que **consulta** a cobrança no Asaas e atualiza a inscrição de acordo com o status de lá. É o *polling* da Figura 6.1, usado como plano B.

1. Acrescente ao serviço um método que faça `GET /v3/payments/{id}`.
2. Mapeie os status do Asaas (`PENDING`, `RECEIVED`, `CONFIRMED`, `OVERDUE`, `REFUNDED`…) para os da inscrição, reaproveitando a mesma regra do webhook.
3. Teste: feche o túnel, confirme um pagamento no sandbox e sincronize.

Dica: para não duplicar a regra, mova para o model `Inscricao` um método como `aplicarStatusAsaas(string $status)`, usado pelos dois caminhos.

11

FECHAMENTO

Síntese, autoavaliação e próximos passos

SÍNTESE DO WORKSHOP

- Integração por API é feita sobre um **contrato**: explore-o no Postman antes de programar.
- Credenciais vivem no **.env**; o código lê a configuração, nunca o segredo diretamente.
- Uma **classe de serviço** isola o sistema externo; o cliente `Http` do Laravel cuida de URL base, cabeçalhos, tempo limite e erros.
- Toda chamada externa pode falhar: decida o que acontece com os seus dados quando falhar (`502` e nada pela metade).
- **Webhooks** invertem os papéis: o serviço externo avisa a nossa API. O endpoint confere o **token**, é **idempotente** e responde 200 rápido.
- Um **túnel** expõe a API local para testes de ponta a ponta; testes com `Http::fake` garantem que a integração continue funcionando.

AUTOAVALIAÇÃO

1. Qual a diferença entre a chave de API do Asaas e o token do webhook? Quem envia cada um, e para quem?
2. O que faz o `throw()` no cliente HTTP, e o que aconteceria no `store` sem ele?
3. Por que o endpoint de webhook responde `200` a um aviso repetido, em vez de um erro?
4. Para que serve o `externalReference` enviado na criação da cobrança?
5. (Discursiva) Imagine que o sistema entra em produção e, num dia de muitas inscrições, o servidor fica fora do ar por duas horas. Descreva o que acontece com os pagamentos feitos nesse período, do ponto de vista do Asaas e da nossa API, e quais mecanismos vistos no workshop (ou que você acrescentaria) garantem que nenhuma inscrição paga fique pendente para sempre.

Glossário

Sandbox — ambiente de testes de um serviço, com a mesma API da produção e sem efeitos reais.

Chave de API — credencial que identifica a aplicação perante o serviço externo.

Classe de serviço — classe que isola uma responsabilidade, como a comunicação com um sistema externo.

Webhook — requisição HTTP enviada por um sistema a uma URL cadastrada, quando um evento acontece.

Polling — consultar repetidamente um sistema para descobrir se algo mudou.

Idempotência — propriedade de uma operação que, repetida, produz o mesmo resultado que executada uma vez.

Túnel — serviço que cria um endereço público e encaminha as requisições a um servidor local.

502 Bad Gateway — status que indica falha de um serviço do qual o servidor depende.

Para ir além

O sistema construído é a base de muitas aplicações reais. Os próximos passos naturais são: **papéis** (hoje, qualquer conta logada cria eventos; com um campo de papel no usuário e um método `create` na `EventoPolicy`, só organizadores o fariam) e tokens com validade, configurada em `config/sanctum.php`; **filas** (*jobs*), para tirar as chamadas ao Asaas do caminho da requisição; **API Resources**, para controlar exatamente quais campos o JSON expõe (o CPF, por exemplo, não deveria sair em qualquer resposta); e um **front-end**, web ou mobile, consumindo a API. Na documentação do Asaas, vale explorar assinaturas recorrentes, parcelamento e o *split* de pagamentos.

Referências

1. ASAAS. **Documentação da API Asaas**: Autenticação; Sandbox; Criar nova cobrança. Disponível em: https://docs.asaas.com. Acesso em: nov. 2026.
2. ASAAS. **Eventos para cobranças**. Disponível em: https://docs.asaas.com/docs/webhook-para-cobrancas. Acesso em: nov. 2026.
3. ASAAS. **Como implementar idempotência em webhooks**. Disponível em: https://docs.asaas.com/docs/como-implementar-idempotencia-em-webhooks. Acesso em: nov. 2026.
4. CLOUDFLARE. **Quick Tunnels**. Cloudflare Tunnel docs. Disponível em: https://developers.cloudflare.com/tunnel/. Acesso em: nov. 2026.
5. LARAVEL. **Laravel 13.x Documentation**: HTTP Client; Configuration; HTTP Tests. Disponível em: https://laravel.com/docs/13.x. Acesso em: nov. 2026.

Material de uso didático, elaborado para o workshop *Desenvolvimento de APIs REST e Integração de Sistemas com Laravel*. As operações com o Asaas descritas aqui usam exclusivamente o ambiente de testes (sandbox).